CSE 209 — Computer Architecture

Chapter 1 · Computer Abstractions, Technology and Performance

Lesson notes · Exam approach · TF practice with solutions · Quick notes (Tushar Sir's portion)

How to Read These Notes

EXAM FOCUS	Exam importance, where it appears in the TF collection
MEMORIZE	Rules, tables and formulas to memorize
TRAP	Common mistakes and exam traps
EXAM-READY ANSWER	Model wording you can write in the exam
NOTE	Extra explanation / clarification
Grey box, monospace	Code, worked tables, derivations and hand-drawn sketches
Centered grey box	Formula to remember / final numerical result
Purple heading / ✓ green	Practice questions / their solutions

Contents

Exam Map — What the TF Papers Ask — performance questions and how much they matter

Lesson 1 — Computers, Abstraction and the Eight Great Ideas — classes, PostPC, great ideas, levels, components, ISA/ABI

Lesson 2 — Defining Performance: Response Time, Throughput, CPU Time — relative performance, clocking, clock-rate problems

Lesson 3 — CPI, Instruction Mix and the Performance Equation — IC × CPI × Tc, weighted CPI, 'how much must CPI improve'

Lesson 4 — Amdahl's Law, the MIPS Pitfall and Benchmarks — Amdahl, parallel speedup limit, MIPS/FLOPS, SPEC (textbook)

Quick Notes — Performance — merged revision notes + key diagrams

> **NOTE** Slide numbers refer to Chapter_01.pdf (Tushar Sir, 28 slides, P&H 5th ed. Chapter 1). "TF p." is the page in CSE209 ALL TF QUES (TF209).pdf — the same file the RS tracker uses. SPEC benchmarks are not on the slides; they are added at the end of Lesson 4 (marked textbook) because two older papers ask them.

Exam Map — What the TF Papers Ask

Every Tushar-Sir-part question that this deck answers (★ = 2019-20 or later):

Session (paper)	Q	What is asked	Marks	Lesson
2023-24 (CSE 209) ★	6(a)	A: 4 GHz, 10 s; B must run in 6 s but needs 1.2× the clock cycles — target clock rate of B?	13	L2
2022-23 (CSE 305) ★	6(d)	P1 2.7 GHz (CPI 2,1,3,2) vs P2 3.2 GHz (2,3,2,3), 1.2×10^6 instr 20/25/40/15 %: cycles, faster, max gain if 40 % improved	15	L3
2022-23 (CSE 305) ★	7(a)	Define response time and throughput. Are they synonymous?	5	L2
2021-22 (CSE 305) ★	5(b)	2 % of runtime not parallelizable, 24 cores: speedup? Can it be made 60× faster?	10	L4
2021-22 (CSE 305) ★	7(b)	FLOPS: misleading as a general metric but useful for a specific part — agree?	16	L4
2021-22 (CSE 305) ★	8(a)	P1 2.7 GHz (CPI 3,3,2) vs P2 3.5 GHz (2,4,1), 200,000 instr 25/60/15 %: cycles, time, CPI of C for 2×	11	L3
2020-21 (CSE 305) ★	10(a)	Briefly explain the eight great ideas in computer architecture	12	L1
2020-21 (CSE 305) ★	12(a)	Programs A and B (mult/add/sub/div CPI 3/1/2/4): which has the higher average CPI?	8	L3
2019-20 (CSE 305) ★	1(a)	Logs of systems A (2 GHz) and B (2.5 GHz): compare performance on Program 1	15	L3
2019-20 (CSE 305) ★	1(b)	Importance of Moore's law; 'improve the most important aspect by x to get x overall' — agree? (math)	12	L4
2018-19 (CSE 305)	7(b)	4 GHz, FP/INT/LS/BR = 160/110/10/16 M, CPI 2/1/5/2: time; FP or LS CPI for 2×; ±15 %/−20 % change	20	L3
2017-18 (CSE 305)	5(a)	Classes A/B/C CPI 1.2/2/2.5; P1 (40,10,16) vs P2 (12,13,40): faster by how much; CPI of B for P1 2×	15	L3
2016-17 (CSE 305)	5(c)	3 GHz, FP/INT/LS/BR = 40/80/90/15 M, CPI 1/1/3/2: time; FP CPI for 2×; +20 %/−10 % CPI change	15	L3
2015-16 (CSE 305)	1(a)	P1 3 GHz CPI 1.5, P2 2.5 GHz CPI 1.0, P3 4 GHz CPI 2.2: highest IPS; cycles & instr in 10 s; clock for −30 % time with +20 % CPI	10	L2
2015-16 (CSE 305)	1(b)	Arith/store/load/branch = 500/50/100/50, cycles 1/5/5/2, 4 GHz: time, CPI; loads halved → speedup, CPI	10	L3
2015-16 (CSE 305)	1(c)	CPI 1/12/5, 2.56 G arith, 1.28 G ld/st, 256 M branch, 2 GHz, divided by 0.7p: time on 1/2/4 processors etc.	15	L3
2015-16 (CSE 305)	2(a)	A: 10 G instr, 4 GHz, CPI 1; B: 8 G, 4 GHz, CPI 1.1 — higher MIPS? faster?	5	L4
2014-15 (CSE 305)	5(b)	2 GHz, FP/INT/LS/BR = 160/110/10/16 M, CPI 2/1/5/2: time; FP CPI for 2×; LS CPI for 2×	15	L3
2013-14 (CSE 305)	5(a)	What is Moore's law? Response time and throughput	6	L2
2013-14 (CSE 305)	5(b)	P1 3 GHz CPI 1/2/3/4 vs P2 2 GHz CPI 2/2/2/2, 200 instr 10/20/50/20 %: cycles, faster, CPI	15	L3
2013-14 (CSE 305)	5(c)	Amdahl's law: 100 s program, division 80 s — improvement in division for 4× overall?	8	L4
2013-14 (CSE 305)	5(d)	A: 10 G instr, 4 GHz, CPI 1; B: 8 G, 4 GHz, CPI 1.1 — higher MIPS? faster?	6	L4
2012-13 (CSE 305)	6(a)	CPU times FP 50 / INT 80 / LS 50 / BR 30 s: % gain for FP −20 %; INT cut for −20 % total; branch only?	9	L4
2011-12 (CSE 305)	1(a)	Components of a computer and the task of each	7.5	L1
2011-12 (CSE 305)	1(c)	A: 250 ps, CPI 2.0; B: 500 ps, CPI 1.2 — which is faster and by how much?	7	L3
2011-12 (CSE 305)	1(d)	10 s program, 50 % FP; FP unit 5× faster — speedup?	6	L4

Session (paper)	Q	What is asked	Marks	Lesson
2011-12 (CSE 305)	1(e)	Problems with measuring performance in MIPS	4.5	L4
Related (RS tracker)	—	Eight great ideas behind given design decisions: 2020-21 Q1, 2016-17 5(a), 2014-15 5(a) — mapped in Lesson 1		L1
Outside (this file)	—	SPEC ratio / geometric mean: 2016-17 5(b), 2020-21 11(a) — solved in Lesson 4 (textbook)		L4

EXAM FOCUS Performance is asked in almost every paper (117 marks since 2019-20, plus 2023-24 6(a)). Two question types dominate: (1) the CPU-time / CPI table — 'which machine is faster, by how much, how much must the CPI of class X improve' (asked 11 times), and (2) Amdahl's law — 'improve one part by n, what is the overall speedup / is k× possible' (6 times). Learn Lesson 3's recipe and Lesson 4's formula first; they are pure marks. Theory (response time vs throughput, great ideas, MIPS pitfall) is short and repeats word for word.

Lesson 1 — Computers, Abstraction and the Eight Great Ideas

Slides 1–13 (Chapter 1 §1.1–1.4)

1. Why Computers Keep Changing (sl. 2)

Progress in computer technology is underpinned by **Moore's Law**: the number of transistors on a chip doubles about every 18–24 months. Cheaper, faster hardware makes new applications feasible (computers in cars, cell phones, the human genome project, the Web, search engines), so computers are pervasive.

2. Classes of Computers (sl. 3–4)

Class	What it is	Design pressure
Personal computer	General purpose, runs a variety of software	Cost / performance trade-off
Server	Network based; small servers up to building-sized	High capacity, performance, reliability
Supercomputer	High-end scientific and engineering calculations	Highest capability; tiny share of the market
Embedded computer	Hidden as a component of a system (car, TV, router)	Stringent power / performance / cost constraints

3. The PostPC Era (sl. 5–6)

- **Personal Mobile Device (PMD)**: battery operated, connected to the Internet, costs hundreds of dollars — smart phones, tablets, electronic glasses. Slide 5's chart: smart-phone sales overtook PC sales around 2011.
- **Cloud computing**: Warehouse-Scale Computers (WSC) provide **Software as a Service (SaaS)**. Part of the software runs on the PMD, part in the cloud (Amazon, Google).

4. What Determines Performance (sl. 7–8)

Hardware or software component	How it affects performance
Algorithm	Number of source-level operations executed
Programming language, compiler, architecture	Number of machine instructions executed per operation
Processor and memory system	How fast instructions are executed
I/O system (including the OS)	How fast I/O operations are executed

5. Eight Great Ideas in Computer Architecture (sl. 9) ★

Great idea	Meaning	Typical example
Design for Moore's Law	Design for where technology will be when the product ships, not where it is when design starts	Plan for a chip that will have 2× the transistors in 2 years
Use abstraction to simplify design	Hide lower-level detail behind a simpler model at each level	ISA hides the implementation; procedures hide their bodies
Make the common case fast	Speed up the frequent case rather than the rare case	Small immediates in instructions; PC-relative branches

Great idea	Meaning	Typical example
Performance via parallelism	Do several operations at the same time	Multicore, 31 adders in a fast multiplier, GPUs
Performance via pipelining	Overlap the steps of successive operations (a special case of parallelism)	5-stage MIPS pipeline; code reordering to avoid load-use stalls
Performance via prediction	Guess and start working; recover if wrong — cheaper on average than waiting	Branch prediction, prefetching
Hierarchy of memories	Fastest, smallest, most expensive memory at the top; slowest, largest, cheapest at the bottom	Registers → caches → DRAM → disk; separate I- and D-caches
Dependability via redundancy	Spare components take over when one fails	RAID disks, ECC memory

> **NOTE** The RS tracker holds three 'which great idea is behind this design' questions (2020-21 Q1, 2016-17 5(a), 2014-15 5(a)). They are answered from this table: single-cycle design replaced (pipelining / make the common case fast), prefetching (prediction), PC-relative branches (make the common case fast), on-chip caches (hierarchy of memories), 31 adders (parallelism), procedures (abstraction), code reordering (pipelining), separate data/instruction caches (hierarchy of memories / parallelism), GPUs after the power wall (parallelism).

6. Below Your Program and Levels of Program Code (sl. 10–11)

- **Application software** — written in a high-level language (HLL).
- **System software** — the **compiler** translates HLL code to machine code; the **operating system** handles input/output, manages memory and storage, and schedules tasks and shares resources.
- **Hardware** — processor, memory, I/O controllers.

Three levels of program code: **high-level language** (close to the problem domain; gives productivity and portability) → **assembly language** (textual representation of instructions) → **hardware representation** (binary digits encoding instructions and data). The compiler does the first step, the assembler the second.

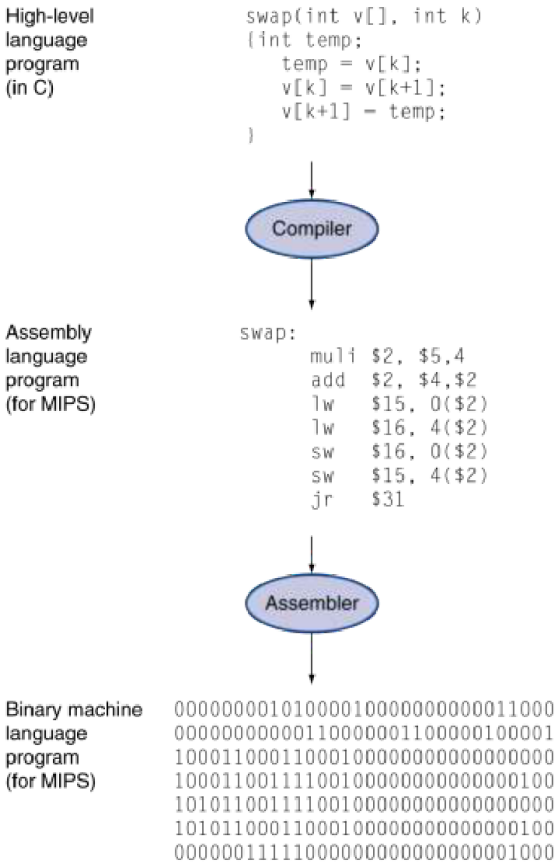


Figure 1.1 — Levels of program code: C swap → compiler → MIPS assembly → assembler → binary (sl. 11)

7. Components of a Computer (sl. 12) ★

The same five components exist in every computer — desktop, server or embedded:

Component	Task
Input	Brings data into the computer (keyboard, mouse, network adapter, disk read)
Output	Sends results out (display, printer, network adapter, disk write)
Memory	Holds programs while they run and the data they need (DRAM, caches); storage devices (disk, flash) keep them permanently
Datapath	Performs the arithmetic and moves data (ALU, registers) — part of the processor
Control	Tells the datapath, memory and I/O what to do according to the instructions — part of the processor

Processor (CPU) = datapath + control. I/O includes user-interface devices (display, keyboard, mouse), storage devices (hard disk, CD/DVD, flash) and network adapters (for communicating with other computers).

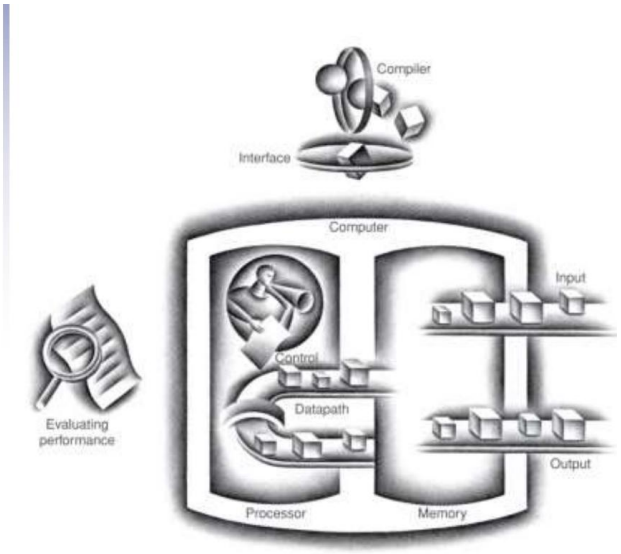


Figure 1.2 — The BIG picture: input, output, memory, datapath, control (sl. 12)

8. Abstractions: ISA, ABI, Implementation (sl. 13)

- Abstraction helps us deal with complexity by hiding lower-level detail.
- **Instruction set architecture (ISA)** — the hardware/software interface: everything a programmer needs to know to write correct machine-language programs (instructions, registers, memory access, I/O).
- **Application binary interface (ABI)** — the ISA (user portion) plus the operating-system interface; it defines binary portability across computers.
- **Implementation** — the hardware that obeys the ISA. Many implementations (different cost/performance) can share one ISA.

Exam Approach

1. Great-ideas questions: name the idea, give one sentence of meaning, then link the given design to it ('…therefore idea X').
2. Components question: list all five, one task each, and say processor = datapath + control.
3. Moore's law: state it (transistors double every 18–24 months) and its importance — designers must design for the technology at shipping time.

Practice — Lesson 1

(a) Actual — 2020-21 10(a). (b) Actual — 2011-12 1(a). (c) Adapted — RS-tracker 2020-21 Q1. (d) Actual part — 2019-20 1(b) (first half).

(a) Briefly explain the eight great ideas in computer architecture. (12)

(b) What are the components of a computer? Mention the task of each of these components. (7.5)

(c) Which great idea is behind: (i) code reordering to avoid using a load result in the next instruction; (ii) different caches for data and instructions; (iii) GPUs gaining popularity after CPUs hit the power limit?

(d) What is the importance of Moore's law?

✓ Solution

(a) Write the eight rows of the table in §5 — name, one-line meaning and one example each. That is the full 12-mark answer; draw the small symbol (rocket, pyramid…) only if time permits.

(b) Table in §7: input, output, memory, datapath, control — with one task each; processor = datapath + control; I/O covers UI devices, storage and network.

(c) (i) Performance via pipelining — reordering removes a load-use stall so the pipeline stays full. (ii) Hierarchy of memories (split L1 caches let fetch and data access proceed in the same cycle — also parallelism). (iii) Performance via parallelism — when the clock can no longer rise because of power, performance must come from many cores working together.

(d) Moore's law (transistor count doubles every 18–24 months) is the engine behind cheaper and faster hardware. Its importance for a designer is the first great idea: a design takes years, so architects must design for the transistor budget that will exist when the product ships, and it explains why new applications become feasible every few years.

Lesson 2 — Defining Performance: Response Time, Throughput, CPU Time

Slides 14–20 (§1.6)

1. What Does 'Best Performance' Mean? (sl. 14)

The airplane example: the Concorde has the highest cruising speed, the Boeing 747 the largest passenger capacity and the largest passengers × mph. 'Best' depends on the metric — for a single passenger, speed (time); for an airline, total passengers moved per hour (throughput).

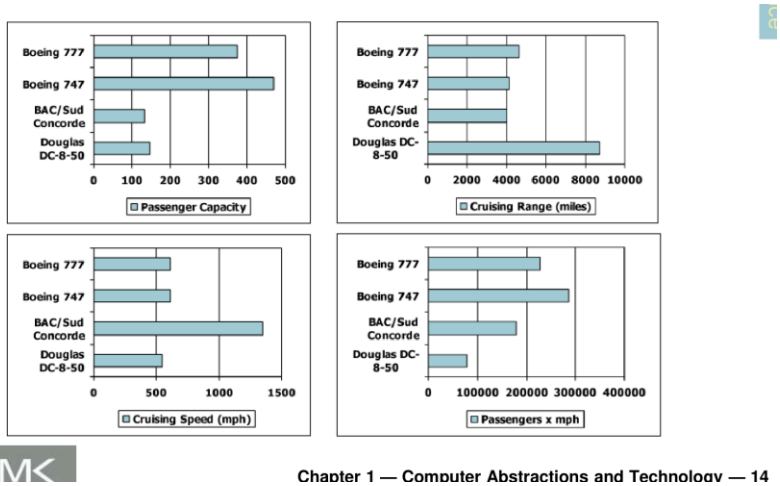


Figure 2.1 — Which airplane has the best performance? It depends on the metric (sl. 14)

2. Response Time and Throughput (sl. 15) ★

Term	Definition	Unit
Response time (execution time, latency)	How long it takes to do one task, from start to completion	seconds per task
Throughput (bandwidth)	Total work done per unit time	tasks / transactions per hour

- **Replacing the processor with a faster one** decreases response time and therefore also increases throughput.
- **Adding more processors** (each task still runs on one processor) increases throughput but does not shorten the response time of one task — unless tasks were waiting in a queue, in which case response time also falls.
- Chapter 1 focuses on response time.

EXAM-READY ANSWER Response time and throughput are **not synonymous**. Response time is the time to complete one task; throughput is the number of tasks completed per unit time. Improving one need not improve the other: adding processors raises throughput while each job takes as long as before; a faster processor improves both.

3. Relative Performance (sl. 16)

$$\textbf{Performance} = \frac{1}{\textbf{Execution time}}$$

$$\textbf{'X is n times faster than Y':}\quad \frac{\text{Perf}_X}{\text{Perf}_Y} = \frac{\text{Time}_Y}{\text{Time}_X} = n$$

Example: a program takes 10 s on A and 15 s on B → $\text{Time}_B / \text{Time}_A = 15/10 = 1.5$, so A is 1.5 times faster than B.

4. Elapsed Time vs CPU Time (sl. 17)

- **Elapsed time** = total response time including everything — processing, I/O, OS overhead, idle time. It determines system performance.
- **CPU time** = time the CPU spends computing for this job; it excludes I/O waiting and other jobs' shares. CPU time = user CPU time + system CPU time.
- Different programs are affected differently by CPU and system performance.

5. CPU Clocking (sl. 18)

Digital hardware is driven by a constant-rate clock. In every cycle data are transferred and computed, then state is updated at the clock edge.

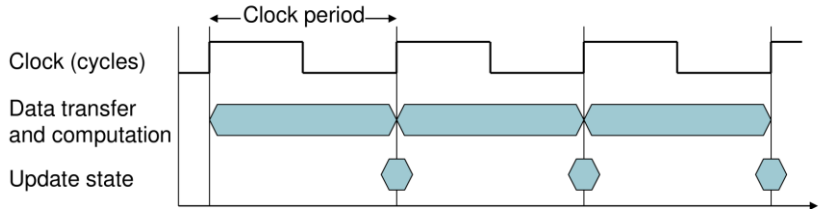


Figure 2.2 — Clock period: data transfer and computation, then state update (sl. 18)

Clock period T_c = duration of one cycle (250 ps = 0.25 ns = 250 × 10^{-12} s)

Clock rate f = 1 / T_c = cycles per second (4.0 GHz = 4000 MHz = 4.0 × 10^9 Hz)

6. CPU Time (sl. 19–20) ★

CPU time = CPU clock cycles × Clock cycle time = CPU clock cycles / Clock rate

Performance improves by reducing the number of clock cycles or by increasing the clock rate. The hardware designer must often trade one against the other: a faster clock may need more (simpler) cycles per task.

Worked example (sl. 20)

Computer A: 2 GHz clock, 10 s CPU time. Design B to run in 6 s; B can have a faster clock but then needs 1.2 × the clock cycles. How fast must B's clock be?

```
Clock cycles_A = CPU time_A × Clock rate_A = 10 s × 2 GHz = 20 × 10^9
Clock rate_B   = Clock cycles_B / CPU time_B = 1.2 × 20 × 10^9 / 6 s
               = 24 × 10^9 / 6 = 4 × 10^9 Hz = 4 GHz
```

TRAP The 1.2 multiplies the number of **cycles**, not the time. First get cycles_A from A's time and clock, then scale, then divide by B's target time.

Exam Approach

1. Write the formula you use first: CPU time = cycles / clock rate.
2. Find the unknown machine's cycle count from the known one (× the given factor).
3. Divide by the target time to get the clock rate; state the answer in GHz.
4. For 'which is faster' compare times (or IPS = clock rate / CPI), and give the ratio 'X is n times faster'.

Practice — Lesson 2

(a) Actual — 2023-24 6(a). (b) Actual — 2015-16 1(a). (c) Actual — 2022-23 (CSE 305) 7(a). (d) Actual — 2013-14 5(a).

(a) Our favorite program runs in 10 s on computer A, which has a 4 GHz clock. Computer B must run it in 6 s; a substantial clock-rate increase is possible but makes B need 1.2 times as many clock cycles as A. What clock rate should we target? (13)

(b) P1: 3 GHz, CPI 1.5; P2: 2.5 GHz, CPI 1.0; P3: 4.0 GHz, CPI 2.2. (i) Highest performance in instructions per second? (ii) Each runs a program in 10 s — number of cycles and instructions? (iii) We reduce the execution time by 30 % but the CPI rises by 20 %. What clock rate is needed? (10)

(c) Define response time and throughput. Are these two terms synonymous? (5)

(d) What is Moore's law? What do you mean by response time and throughput? (6)

✓ Solution

(a)

```
Cycles_A = 10 s × 4 × 10^9 = 40 × 10^9 cycles
Cycles_B = 1.2 × 40 × 10^9 = 48 × 10^9 cycles
Clock_B  = 48 × 10^9 / 6 s = 8 × 10^9 Hz
```

Target clock rate for B = 8 GHz (2× A's clock for 1.67× the speed)

(b) Instructions per second = clock rate / CPI.

	P1	P2	P3
(i) IPS = f / CPI	3/1.5 = 2.0 × 10^9	2.5/1.0 = 2.5 × 10^9	4/2.2 = 1.82 × 10^9
(ii) cycles = 10 s × f	3.0 × 10^10	2.5 × 10^10	4.0 × 10^10
(ii) instructions = cycles / CPI	2.0 × 10^10	2.5 × 10^10	1.82 × 10^10
(iii) new f = f × 1.2 / 0.7	5.14 GHz	4.29 GHz	6.86 GHz

(i) **P2** has the highest performance (2.5 × 10^9 instructions/s). (iii) Time = IC × CPI / f. New time = 0.7 × old, new CPI = 1.2 × old, IC unchanged → 0.7 = 1.2 / (f_{new}/f_{old}) → f_{new} = (1.2/0.7) f_{old} = 1.714 f_{old}.

(c) Model answer in the EXAM-READY box of §2: not synonymous; define both; a faster CPU improves both, extra processors improve only throughput.

(d) Moore's law: the number of transistors on an integrated circuit doubles roughly every 18–24 months (Lesson 1). Response time and throughput: §2.

Lesson 3 — CPI, Instruction Mix and the Performance Equation

Slides 21–25 (§1.6)

EXAM FOCUS The most-asked numerical type of the ART part: 2022-23 6(d), 2021-22 8(a), 2020-21 12(a), 2019-20 1(a), 2018-19 7(b), 2017-18 5(a), 2016-17 5(c), 2015-16 1(b)–(c), 2014-15 5(b), 2013-14 5(b), 2011-12 1(c). Every one is solved below.

1. Instruction Count and CPI (sl. 21)

$$\text{Clock cycles} = \text{Instruction count} \times \text{Cycles per instruction}$$
$$\text{CPU time} = IC \times CPI \times T_c = IC \times CPI / \text{Clock rate}$$

- **Instruction count (IC)** is determined by the program, the ISA and the compiler.
- **Average CPI** is determined by the CPU hardware. If instructions have different CPIs, the average CPI depends on the instruction mix.

2. CPI Example (sl. 22)

A: cycle time 250 ps, CPI 2.0; B: cycle time 500 ps, CPI 1.2; same ISA (same IC = I). Which is faster, and by how much?

```
CPU time_A = I × 2.0 × 250 ps = I × 500 ps
CPU time_B = I × 1.2 × 500 ps = I × 600 ps
Time_B / Time_A = 600 / 500 = 1.2   →  A is 1.2 times faster
```

3. CPI in More Detail — Weighted Average (sl. 23)

$$\text{Clock cycles} = \Sigma_i (CPI_i \times IC_i)$$
$$CPI = \text{Clock cycles} / IC = \Sigma_i CPI_i \times (IC_i / IC) \quad (IC_i/IC = \text{relative frequency})$$

4. Worked Example: Two Code Sequences (sl. 24)

Class	A	B	C
CPI for class	1	2	3
IC in sequence 1	2	1	2
IC in sequence 2	4	1	1

```
Sequence 1: IC = 5, cycles = 2×1 + 1×2 + 2×3 = 10, CPI = 10/5 = 2.0
Sequence 2: IC = 6, cycles = 4×1 + 1×2 + 1×3 = 9,  CPI = 9/6  = 1.5
Sequence 2 executes MORE instructions but is FASTER (9 < 10 cycles).
```

TRAP Never compare machines by IC alone or by CPI alone. Only the product IC × CPI × T_c (time) decides.

5. Performance Summary — The BIG Picture (sl. 25)

CPU time = (Instructions / Program) × (Clock cycles / Instruction) × (Seconds / Clock cycle)

Factor	Affects
Algorithm	IC, possibly CPI
Programming language	IC, CPI
Compiler	IC, CPI
Instruction set architecture	IC, CPI, T_c

6. The 'How Much Must CPI Improve?' Recipe

1. Compute total cycles = $\Sigma\ IC_i \times CPI_i$ (keep 10^6 as a common unit).
2. Target cycles = old cycles / k for 'k times faster' (same clock).
3. Cycles that the improved class may use = target − (cycles of all other classes).
4. If that number is ≤ 0 → **impossible**, even with CPI = 0 for that class. Otherwise new CPI = allowed cycles / IC of the class; improvement factor = old CPI / new CPI.

Exam Approach

1. Make a table: class | IC | CPI | IC × CPI. Sum the last column.
2. Time = cycles / clock rate. With different clocks compare times, not cycles.
3. Average CPI = total cycles / total IC.
4. For CPI-improvement questions use the recipe in §6 and say 'impossible' with the negative number as proof.
5. For percentage changes multiply each CPI by (1 ± p) before summing; report new time, change in % and speedup.

Practice — Lesson 3

(a) Actual — 2018-19 7(b) (same table as 2014-15 5(b), 2 GHz). (b) Actual — 2016-17 5(c). (c) Actual — 2022-23 (CSE 305) 6(d). (d) Actual — 2021-22 8(a). (e) Actual — 2019-20 1(a). (f) Actual — 2017-18 5(a).

(a) A 4 GHz processor runs a program with FP / INT / L/S / BRANCH = 160 / 110 / 10 / 16 million instructions and CPI 2 / 1 / 5 / 2. (i) Execution time? (ii) How much must the CPI of FP improve for the program to run 2× faster? (iii) Of L/S? (iv) Change in time if CPI of INT and BRANCH rise 15 % and CPI of FP and L/S fall 20 %? (20)

(b) 3 GHz; FP / INT / L/S / BRANCH = 40 / 80 / 90 / 15 million, CPI 1 / 1 / 3 / 2. (i) Execution time. (ii) FP CPI for 2×? (iii) Change if CPI of INT and FP rise 20 % and of L/S and Branch fall 10 %. (15)

(c) P1: 2.7 GHz, CPI (A, B, C, D) = 2, 1, 3, 2; P2: 3.2 GHz, CPI 2, 3, 2, 3. 1.2 × 10^6 instructions: 20 % A, 25 % B, 40 % C, 15 % D. (i) Clock cycles on each. (ii) Which is faster, by how much? (iii) 40 % of the code is improved — maximum possible improvement factor? (15)

(d) 200,000 instructions: 25 % A, 60 % B, 15 % C. P1 2.7 GHz, CPI 3/3/2; P2 3.5 GHz, CPI 2/4/1. (i) Total cycles, (ii) execution times, which is faster, (iii) CPI improvement of class C for 2× in the better one. (11)

(e) Logs: on A (2 GHz) Program 1 uses Mul 4, Mul 4, Div 5, Div 5, Add 2 cycles; on B (2.5 GHz) Mul 5, Mul 5, Div 6, Div 6, Add 3 cycles (other entries belong to other programs or OS I/O). Compare the performance on Program 1. (15)

(f) CPIs of A, B, C = 1.2, 2, 2.5. P1 has 40, 10, 16 and P2 12, 13, 40 instructions of A, B, C. (i) Which executes faster, by how much? (ii) CPI improvement of B for P1 to run 2× faster? (15)

✓ Solution

(a)

Class	IC (×10^6)	CPI	Cycles (×10^6)	(iv) new CPI	(iv) cycles
FP	160	2	320	1.6	256
INT	110	1	110	1.15	126.5
L/S	10	5	50	4.0	40
BRANCH	16	2	32	2.3	36.8
Total	296		512		459.3

```
(i)   T = 512 × 10^6 / 4 × 10^9 = 0.128 s              (2014-15 5(b) at 2 GHz: 0.256 s)
(ii)  2× faster → 256 × 10^6 cycles. Others use 110 + 50 + 32 = 192 → FP may use 64
      CPI_FP = 64 / 160 = 0.4  → FP CPI must improve 2 / 0.4 = 5 times
(iii) Others use 320 + 110 + 32 = 462 > 256 → impossible: even CPI_LS = 0 is not enough
(iv)  T = 459.3 × 10^6 / 4 × 10^9 = 0.1148 s → 10.3 % less time (speedup 1.115)
```

(b) Cycles = 40×1 + 80×1 + 90×3 + 15×2 = 420 × 10^6 → T = 420 × 10^6 / 3 × 10^9 = **0.14 s**. (ii) 2× → 210 × 10^6 cycles; the non-FP classes alone need 80 + 270 + 30 = 380 × 10^6 > 210 → **impossible**. (iii) New cycles = 40×1.2 + 80×1.2 + 90×2.7 + 15×1.8 = 48 + 96 + 243 + 27 = 414 × 10^6 → T = 0.138 s: **improved by 1.43 %** (speedup 1.014).

(c) IC: A 240,000; B 300,000; C 480,000; D 180,000.

```
P1 cycles = 240k×2 + 300k×1 + 480k×3 + 180k×2 = 2.58 × 10^6   → T1 = 2.58e6 / 2.7e9 = 0.956 ms
P2 cycles = 240k×2 + 300k×3 + 480k×2 + 180k×3 = 2.88 × 10^6   → T2 = 2.88e6 / 3.2e9 = 0.900 ms
P2 is faster: 0.956 / 0.900 = 1.06 times
(iii) Amdahl: if 40 % of the time is improved infinitely, Speedup_max = 1 / (1 − 0.4) = 1.67
```

(d) A = 50,000, B = 120,000, C = 30,000 instructions.

```
P1 cycles = 50k×3 + 120k×3 + 30k×2 = 570,000 → T = 570,000 / 2.7 GHz = 211.1 µs
P2 cycles = 50k×2 + 120k×4 + 30k×1 = 610,000 → T = 610,000 / 3.5 GHz = 174.3 µs
P2 is faster by 211.1 / 174.3 = 1.21
(iii) 2× in P2 → 305,000 cycles; A and B alone need 100k + 480k = 580k > 305k → impossible
```

(e) CPU time counts only Program 1's own operations — ignore the other programs and the OS I/O overhead lines.

```
A: cycles = 4 + 4 + 5 + 5 + 2 = 20 → T_A = 20 / 2 GHz   = 10 ns
B: cycles = 5 + 5 + 6 + 6 + 3 = 25 → T_B = 25 / 2.5 GHz = 10 ns
Performance_A / Performance_B = T_B / T_A = 1 → equal performance on Program 1
```

> **TRAP** 2019-20 1(a) is built to trap you: B has the faster clock but needs more cycles; and adding the I/O lines (elapsed time) would give a different answer. CPU time is the right measure for 'performance of the processor on Program 1' — say so.

(f) Cycles P1 = 40×1.2 + 10×2 + 16×2.5 = 48 + 20 + 40 = 108; P2 = 12×1.2 + 13×2 + 40×2.5 = 14.4 + 26 + 100 = 140.4. Same machine (same clock) → P1 is faster by 140.4 / 108 = **1.3 times**. (ii) P1 2× faster → 54 cycles; A and C alone use 88 → **impossible**.

More Solved TF Questions — Lesson 3

2013-14 5(b) 200 instructions: A 20, B 40, C 100, D 40. P1 (3 GHz, CPI 1/2/3/4): 20 + 80 + 300 + 160 = **560 cycles**, CPI = 2.8, T = 560 / 3 GHz = 186.7 ns. P2 (2 GHz, CPI 2): **400 cycles**, CPI = 2.0, T = 200 ns. P1 is faster by 200 / 186.7 = 1.07 (fewer cycles do not win alone — P2's clock is slower).

2015-16 1(b) Cycles = 500×1 + 50×5 + 100×5 + 50×2 = 1350; IC = 700. (i) T = 1350 / 4 GHz = **337.5 ns**. (ii) CPI = 1350/700 = **1.93**. (iii) Loads halved → 50 loads: cycles = 1100, IC = 650 → CPI = 1.69; speedup = 1350/1100 = **1.23**.

2015-16 1(c) Per processor: arithmetic and load/store counts are divided by 0.7p; branches stay 256 M. Clock 2 GHz.

p	Arith (×10^9)	L/S (×10^9)	Cycles = A×1 + LS×12 + 0.256×5	Time	Speedup	(ii) CPI_A = 2
1	2.560	1.280	2.56 + 15.36 + 1.28 = 19.2 × 10^9	9.60 s	1	10.88 s
2	1.829	0.914	1.829 + 10.971 + 1.28 = 14.08 × 10^9	7.04 s	1.36	7.95 s
4	0.914	0.457	0.914 + 5.486 + 1.28 = 7.68 × 10^9	3.84 s	2.50	4.30 s

(iii) One processor must finish in 3.84 s = 7.68 × 10^9 cycles: 2.56 + 1.28 × CPI_{LS} + 1.28 = 7.68 → CPI_{LS} = **3** (reduce from 12 to 3, i.e. by 75 %).

2020-21 12(a) Program A: IC = 2+1+4+2 = 9, cycles = 2×3 + 1×1 + 4×2 + 2×4 = 23 → CPI = 2.56. Program B: IC = 10, cycles = 3×3 + 2×1 + 3×2 + 2×4 = 25 → CPI = 2.50. **Program A** has the higher average CPI.

2011-12 1(c) Exactly slide 22: A takes I × 2.0 × 250 ps = 500I ps, B takes I × 1.2 × 500 ps = 600I ps → **A is 1.2 times faster**.

Lesson 4 — Amdahl's Law, the MIPS Pitfall and Benchmarks

Slides 26–28 (§1.9–1.10) + SPEC (textbook §1.10)

1. Pitfall: Amdahl's Law (sl. 26) ★

Pitfall: improving one aspect of a computer and expecting a proportional improvement in overall performance.

$$T_{improved} = T_{affected} / \text{improvement factor} + T_{unaffected}$$

$$\text{Speedup} = 1 / ((1 - f) + f / n) \qquad f = \text{fraction of time improved, } n = \text{factor}$$

$$\text{Upper limit } (n \to \infty): \ \text{Speedup}_{max} = 1 / (1 - f)$$

Slide example

```
Multiply takes 80 s of a 100 s program. How much must multiply improve for 5× overall?
Target 100/5 = 20 s:   20 = 80/n + 20   →  80/n = 0   →  can't be done!
For 4× overall (2013-14 5(c), with division instead of multiply):
   25 = 80/n + 20  →  80/n = 5  →  n = 16   (division must become 16× faster)
```

Corollary: **make the common case fast** — the unaffected part sets the ceiling.

EXAM-READY ANSWER 2019-20 1(b): 'To increase overall performance by x, identify the most important aspect and improve it by x' — **Disagree**. If that aspect takes a fraction f < 1 of the time, Speedup = 1/((1 − f) + f/x), which is less than x for every x > 1 because the unimproved part (1 − f) still takes the same time. Example: f = 0.8, x = 4 → Speedup = 1/(0.2 + 0.2) = 2.5, not 4; and for x = 5 the overall limit is only 1/0.2 = 5 as x → ∞. Overall speedup x is reached only if f = 1.

Amdahl and parallel processors

With p processors the parallel fraction f runs p times faster: Speedup = 1 / ((1 − f) + f/p). The serial fraction limits the speedup to 1/(1 − f) no matter how many cores are added.

2. Pitfall: MIPS as a Performance Metric (sl. 27) ★

$$\text{MIPS} = \text{Instruction count} / (\text{Execution time} \times 10^6) = \text{Clock rate} / (\text{CPI} \times 10^6)$$

MIPS (millions of instructions per second) does not account for:

- differences in ISAs between computers (one machine may need more instructions for the same work);
- differences in complexity between instructions (a 'simple' instruction counts the same as a complex one);
- CPI varies between programs on the same CPU, so one machine has no single MIPS rating.

Hence a machine with a higher MIPS rating can be slower. Execution time is the only reliable measure.

NOTE FLOPS (floating-point operations per second) has the same weakness as MIPS: it ignores integer, memory and branch work and depends on the FP instruction set. It is misleading as a general metric, but it is a fair measure of one part — the FP unit — or of FP-dominated programs (scientific computing, GPUs).

3. Concluding Remarks (sl. 28)

- Cost/performance is improving because of underlying technology development.
- Hierarchical layers of abstraction exist in both hardware and software.
- The ISA is the hardware/software interface.
- **Execution time is the best performance measure.**
- Power is a limiting factor — so parallelism is now used to improve performance.

4. Benchmarks: SPEC Ratio (textbook §1.10 — not on the slides)

SPEC CPU benchmarks report, for each program i, SPECratio_i = Reference time_i / Execution time_i. The overall figure is the **geometric mean**: $(\Pi\ \text{SPECratio}_i)^{1/n}$. The geometric mean gives the same ranking whichever machine is used as the reference.

Exam Approach

1. Amdahl: write $T_{new} = T_{aff}/n + T_{unaff}$; solve for the unknown; if it needs 0 or negative time → 'impossible'.
2. Parallel speedup: $S = 1/((1 - f) + f/p)$; limit $1/(1 - f)$ answers 'is k× possible?'.
3. MIPS rating: compute $\text{MIPS} = f/(\text{CPI} \times 10^6)$ AND execution time; show they rank the machines differently.
4. SPEC: ratio per program, then geometric mean; for a missing time solve $\text{GM}^n = \Pi$ ratios.

Practice — Lesson 4

(a) Actual — 2021-22 5(b). (b) Actual — 2012-13 6(a). (c) Actual — 2011-12 1(d). (d) Actual — 2015-16 2(a) = 2013-14 5(d). (e) Actual — 2021-22 7(b). (f) Actual — 2016-17 5(b) and 2020-21 11(a) (Outside-Notes questions, textbook).

(a) 2 % of a program's runtime is not parallelizable. It runs on 24 cores with no overheads. (i) Parallel speedup? (ii) Is it possible to make the program 60 times faster? (10)

(b) CPU times: FP 50 s, INT 80 s, L/S 50 s, Branch 30 s (total 210 s). (i) % total reduction if FP time drops 20 %? (ii) If only INT improves, by how much must INT time fall for a 20 % total reduction? (iii) Can branch alone give 20 %? (9)

(c) A program runs 10 s and spends 50 % in FP instructions; the FP unit becomes 5× faster. Speedup? (6)

(d) A: 10 billion instructions, 4 GHz, CPI 1. B: 8 billion, 4 GHz, CPI 1.1. (i) Which has the higher MIPS rating? (ii) Which is faster? (5/6)

(e) Your partner says FLOPS is misleading for general performance but useful for a specific part or type of processor. Agree or disagree? (16)

(f) (i) Find the SPEC ratio of the server from the 12-program SPECint2006 table. (ii) SPECBM101: geometric mean 28.74; times of programs 2–6 are 527, 586, 109, 470, 275 s (refs 10490, 12100, 20720, 7020, 6900); ref of program 1 is 9770 s. Execution time of program 1?

✓ Solution

(a) (i) S = 1 / (0.02 + 0.98/24) = 1 / (0.02 + 0.04083) = **16.44**. (ii) Limit = 1/0.02 = **50** < 60 → impossible with this program, however many cores. It could be reached only by shrinking the serial part (re-designing the algorithm so that < 1/60 ≈ 1.67 % is serial) or by making the serial part itself faster (a faster core).

(b) (i) FP saves 0.2 × 50 = 10 s → 200 s: total reduced by 10/210 = **4.76 %**. (ii) 20 % of 210 = 42 s must come from INT: 42/80 = **52.5 %** reduction of INT time. (iii) Branch takes only 30 s < 42 s → **no**, even if branch time became zero.

(c) T_{new} = 5/5 + 5 = 6 s → speedup = 10/6 = **1.67**.

(d) MIPS_A = 4 × 10^9 / (1 × 10^6) = 4000; MIPS_B = 4 × 10^9 / (1.1 × 10^6) = 3636 → **A has the higher MIPS**. Time_A = 10 × 10^9 × 1 / 4 × 10^9 = 2.5 s; Time_B = 8 × 10^9 × 1.1 / 4 × 10^9 = 2.2 s → **B is faster** (1.14×). MIPS ranks them the wrong way because B executes fewer instructions.

(e) Agree. FLOPS counts only floating-point operations per second, so (1) programs dominated by integer, memory or branch work are not measured at all; (2) different ISAs offer different FP operations (a fused multiply-add counts as one or two), so FLOPS of different machines are not comparable; (3) like MIPS (sl. 27), FLOPS varies from program to program on the same machine. Execution time of real programs is the only general measure. But for a specific part — the FP unit or a GPU — or for FP-heavy workloads (matrix multiply, scientific simulation, deep learning), peak and sustained FLOPS directly tell how fast that part does its job, so it is a useful specialised metric.

(f) (i) Ratios: perlbench 41.40, bzip2 25.73, gcc 38.70, mcf 68.57, gobmk 31.13, hmmer 88.86, sjeng 35.59, libquantum 10360, h264ref 58.39, omnetpp 52.52, astar 36.00, xalancbmk 80.23 → geometric mean = $(\Pi \text{ ratios})^{1/12}$ = **73.7**. (ii) Ratios 2–6: 19.91, 20.65, 190.09, 14.94, 25.09. $28.74^6 = \text{ratio}_1 \times \Pi(\text{others}) \rightarrow \text{ratio}_1 = 19.25 \rightarrow T_1 = 9770 / 19.25 =$ **507.6 s**.

Quick Notes — Performance

Revision points for Lessons 1–4.

L1 · Abstraction and Great Ideas

- Classes: PC (cost/perf), server (capacity, reliability), supercomputer (highest capability), embedded (power/perf/cost). PostPC: PMD + cloud (WSC, SaaS).
- Performance factors: algorithm → ops; language/compiler/ISA → instructions per op; processor+memory → speed of instructions; I/O+OS → I/O speed.
- 8 ideas: Moore's law · abstraction · common case fast · parallelism · pipelining · prediction · memory hierarchy · redundancy.
- Five components: input, output, memory, datapath, control (CPU = datapath + control).
- ISA = HW/SW interface; ABI = ISA + OS interface; implementation = hardware obeying the ISA.

L2 · Defining Performance

- Response time = time for one task; throughput = tasks per unit time — not synonymous.
- Perf = 1/T; X is n× faster than Y ⇔ $T_Y / T_X = n$. CPU time (user + system) excludes I/O and other jobs.
- CPU time = cycles × T_c = cycles / f. Example: A 4 GHz 10 s, B 6 s with 1.2× cycles → 8 GHz.

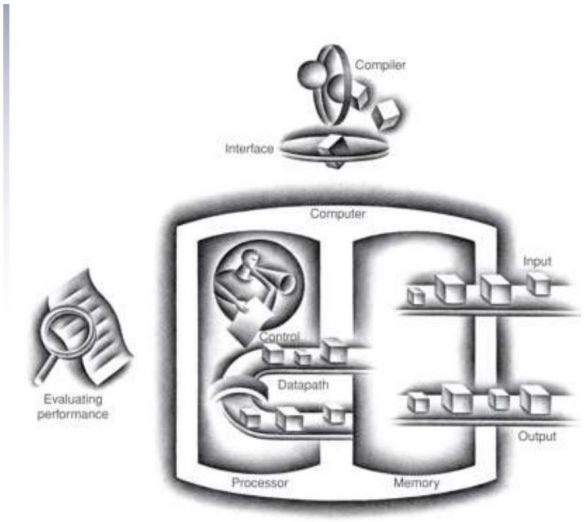
L3 · CPI and the Performance Equation

- CPU time = IC × CPI × T_c; cycles = Σ $CPI_i \times IC_i$; CPI = cycles / IC.
- IC: program, ISA, compiler. CPI: hardware + mix. T_c: hardware (ISA affects all three).
- 'k× faster by improving class X': allowed cycles = old/k − others; ≤ 0 → impossible; else CPI_{new} = allowed / IC_X.
- 2019-20 1(a): A 20 cycles/2 GHz = 10 ns, B 25/2.5 GHz = 10 ns → equal.

L4 · Amdahl, MIPS, SPEC

- $T_{new} = T_{aff}/n + T_{unaff}$; $S = 1/((1-f) + f/n)$; $S_{max} = 1/(1-f)$. 80 s of 100 s: 5× impossible, 4× needs 16×.
- 2 % serial on 24 cores → 16.44; limit 50 → 60× impossible.
- MIPS = f/(CPI × 10^6): ignores ISA differences, instruction complexity, program variation. FLOPS has the same flaws but is fine for FP units.
- SPECratio = ref/exec; overall = geometric mean.

Key Diagrams to Reproduce



Components of a computer — the BIG picture (sl. 12) — same as Fig. 1.2